

Build Your Own Cloud

Dev, DevOps & Ops

Bjørn Kristian Punsvik
November, 2025



```
bk@crayon:~$ whoami
```

```
-----  
Name:      Bjørn Kristian Punsvik  
Role:      Software Developer  
Position:  Senior Consultant  
Location:  Trondheim  
Uptime:    30 years  
Experience: 5 years  
Education: B.Sc. Computer Science, NTNU
```

```
bk@crayon:~$ ps -a
```

```
-----  
PID    COMMAND                                TIME  
1001    /usr/bin/linux-server-admin            9y  
1002    /usr/bin/kubernetes                    5y  
1003    /usr/bin/aws                           2y  
1004    /usr/bin/azure                          3y  
1005    /usr/bin/devops                         2y
```

Development





THE TWELVE-FACTOR APP

<https://12factor.net>



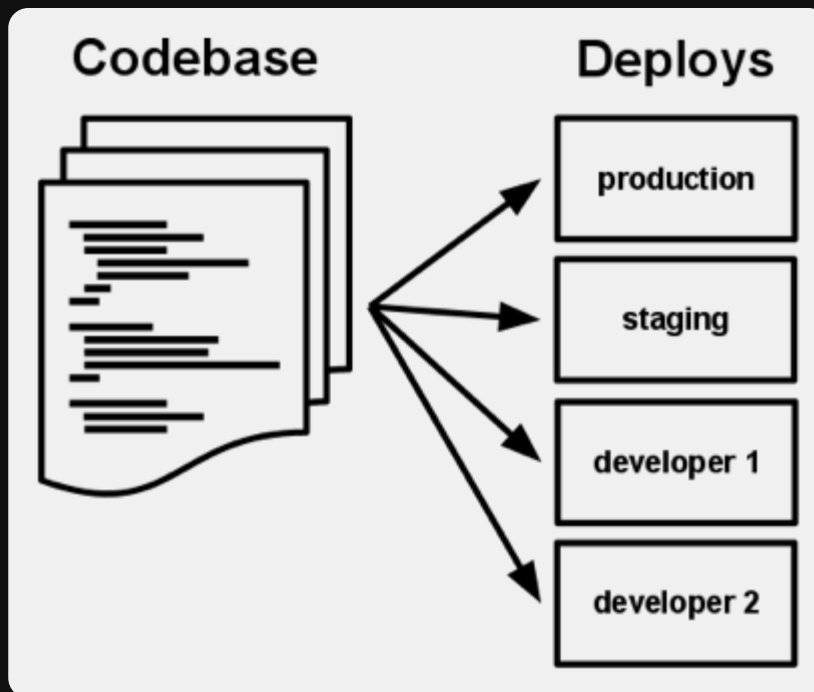
THE TWELVE-FACTOR APP

- I. Codebase
- II. Dependencies
- III. Config
- IV. Backing services
- V. Build, release, run
- VI. Processes
- VII. Port binding
- VIII. Concurrency
- IX. Disposability
- X. Dev/prod parity
- XI. Logs
- XII. Admin processes



I. Codebase

- II. Dependencies
- III. Config
- IV. Backing services
- V. Build, release, run
- VI. Processes
- VII. Port binding
- VIII. Concurrency
- IX. Disposability
- X. Dev/prod parity
- XI. Logs
- XII. Admin processes



One codebase tracked in revision control, many deploys



I. Codebase

II. Dependencies

III. Config
IV. Backing services
V. Build, release, run
VI. Processes
VII. Port binding
VIII. Concurrency
IX. Disposability
X. Dev/prod parity
XI. Logs
XII. Admin processes

A twelve-factor app never relies on implicit existence of system-wide packages.

Explicitly declare and isolate dependencies



- I. Codebase
- II. Dependencies

An app's *config* is everything that is likely to vary between **deploys** (staging, production, developer environments, etc).

III. Config

This includes:

- IV. Backing services
- V. Build, release, run
- VI. Processes
- VII. Port binding
- VIII. Concurrency
- IX. Disposability
- X. Dev/prod parity
- XI. Logs
- XII. Admin processes

- Resource handles to the database, Memcached, and other **backing services**
- Credentials to external services such as Amazon S3 or Twitter
- Per-deploy values such as the canonical hostname for the deploy

The twelve-factor app stores config in *environment variables* (often shortened to *env vars* or *env*).

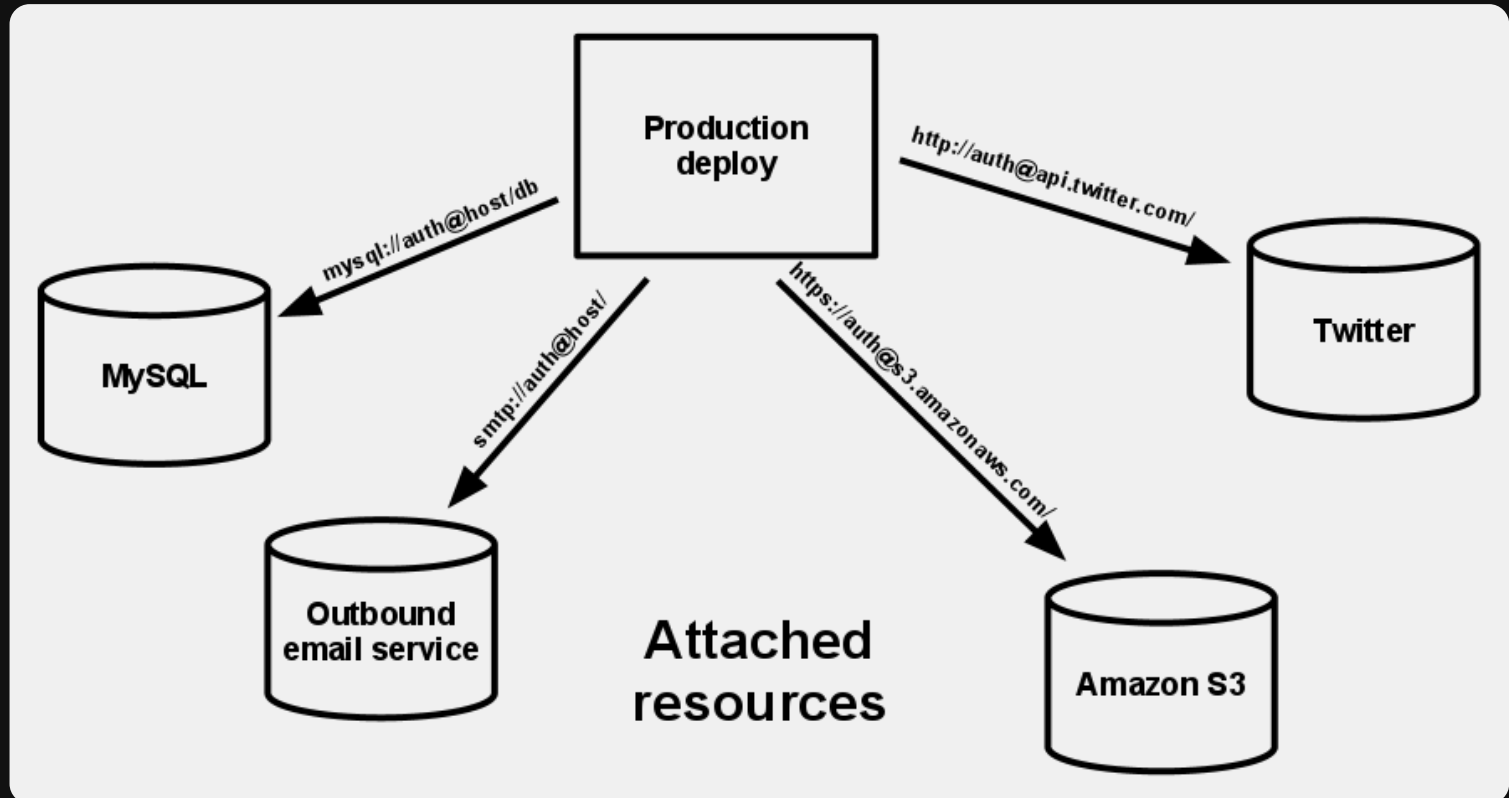
Store config in the environment



- I. Codebase
- II. Dependencies
- III. Config

IV. Backing services

- V. Build, release, run
- VI. Processes
- VII. Port binding
- VIII. Concurrency
- IX. Disposability
- X. Dev/prod parity
- XI. Logs
- XII. Admin processes



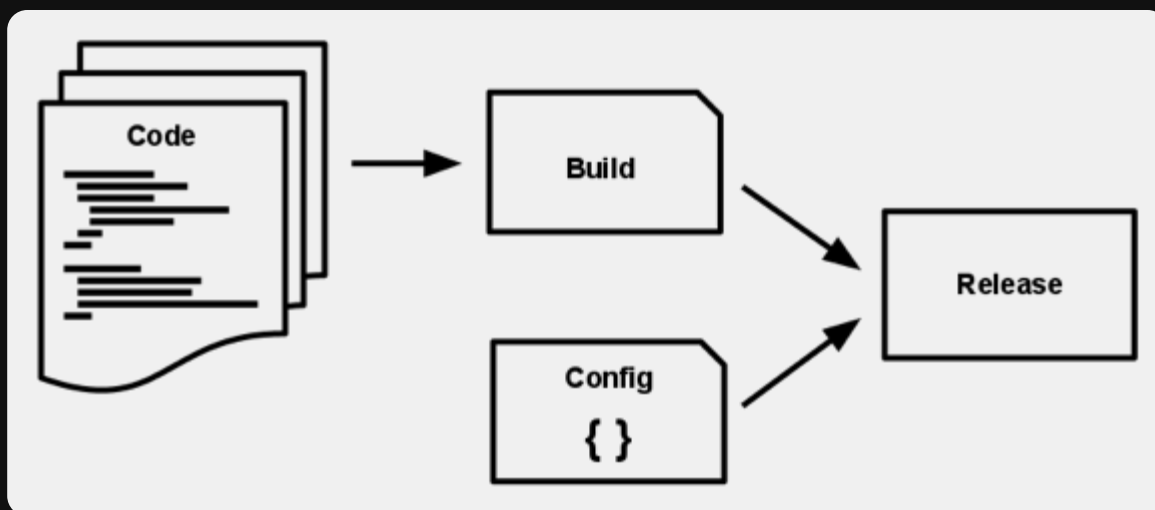
Treat backing services as attached resources



- I. Codebase
- II. Dependencies
- III. Config
- IV. Backing services

V. Build, release, run

- VI. Processes
- VII. Port binding
- VIII. Concurrency
- IX. Disposability
- X. Dev/prod parity
- XI. Logs
- XII. Admin processes



Strictly separate build and run stages



- I. Codebase
- II. Dependencies
- III. Config
- IV. Backing services
- V. Build, release, run

The app is executed in the execution environment as one or more processes.

VI. Processes

Twelve-factor processes are stateless and share-nothing. Any data that needs to persist must be stored in a stateful **backing service**, typically a database.

- VII. Port binding
- VIII. Concurrency
- IX. Disposability
- X. Dev/prod parity
- XI. Logs
- XII. Admin processes

Execute the app as one or more stateless processes



- I. Codebase
- II. Dependencies
- III. Config
- IV. Backing services
- V. Build, release, run
- VI. Processes

VII. Port binding

The twelve-factor app is completely *self-contained* and does not rely on runtime injection of a webserver into the execution environment to create a web-facing service. The web app exports HTTP as a service by *binding to a port*, and listening to requests coming in on that port.

- VIII. Concurrency
- IX. Disposability
- X. Dev/prod parity
- XI. Logs
- XII. Admin processes

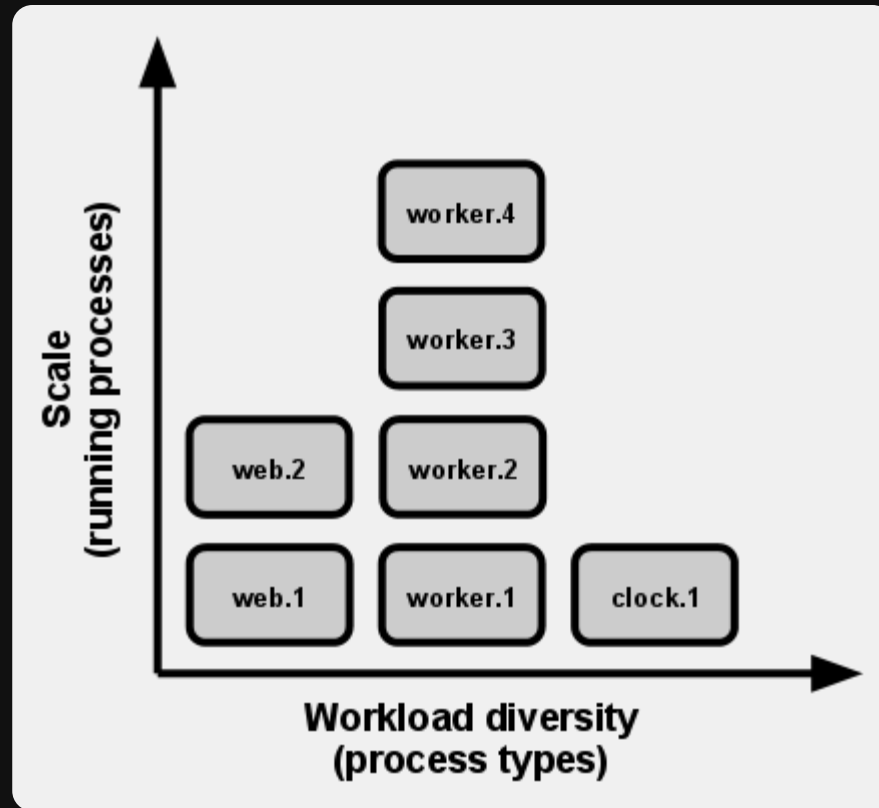
Export services via port binding



- I. Codebase
- II. Dependencies
- III. Config
- IV. Backing services
- V. Build, release, run
- VI. Processes
- VII. Port binding

VIII. Concurrency

- IX. Disposability
- X. Dev/prod parity
- XI. Logs
- XII. Admin processes



Scale out via the process model



- I. Codebase
- II. Dependencies
- III. Config
- IV. Backing services
- V. Build, release, run
- VI. Processes
- VII. Port binding
- VIII. Concurrency

The twelve-factor app's processes are disposable, meaning they can be started or stopped at a moment's notice.

IX. Disposability

Processes should strive to minimize startup time. Ideally, a process takes a few seconds from the time the launch command is executed until the process is up and ready to receive requests or jobs.

- X. Dev/prod parity
- XI. Logs
- XII. Admin processes

Maximize robustness with fast startup and graceful shutdown



- I. Codebase
- II. Dependencies
- III. Config
- IV. Backing services
- V. Build, release, run
- VI. Processes
- VII. Port binding
- VIII. Concurrency
- IX. Disposability\

The twelve-factor app is designed for **continuous deployment** by keeping the gap between development and production small.

	Traditional app	Twelve-factor app
Time between deploys	Weeks	Hours
Code authors vs code deployers	Different people	Same people
Dev vs production environments	Divergent	As similar as possible

X. Dev/prod parity

Resists the urge to use different backing services between development and production

- XI. Logs
- XII. Admin processes

Keep development, staging, and production as similar as possible



- I. Codebase
- II. Dependencies
- III. Config
- IV. Backing services
- V. Build, release, run
- VI. Processes
- VII. Port binding
- VIII. Concurrency
- IX. Disposability
- X. Dev/prod parity\

Logs provide visibility into the behavior of a running app.

A twelve-factor app never concerns itself with routing or storage of its output stream.

XI. Logs

- XII. Admin processes

Treat logs as event streams



- I. Codebase
- II. Dependencies
- III. Config
- IV. Backing services
- V. Build, release, run
- VI. Processes
- VII. Port binding
- VIII. Concurrency
- IX. Disposability
- X. Dev/prod parity
- XI. Logs

developers will often wish to do one-off administrative or maintenance tasks for the app, such as:

- Running database migrations
- Running a console (also known as a **REPL** shell) to run arbitrary code or inspect the app's models against the live database
- Running one-time scripts committed into the app's repo (e.g. `php scripts/fix_bad_records.php`)

XII. Admin processes

Run admin/management tasks as one-off processes



Dotnet

10.0.100-rc.2

```
winget install Microsoft.DotNet.SDK.Preview  
winget install Microsoft.DotNet.Runtime.Preview  
winget install Microsoft.DotNet.AspNetCore.Preview
```

```
dotnet new webapp -lang C# -f net10.0 -n byoc -o . --exclude-launch-settings --no-https
```



.gitignore

```
# Downloaded content
```

```
# Built content
```

```
# User files
```

```
# Temp files
```

```
# Secrets
```



.gitignore

```
# Downloaded content
```

```
# Built content
```

```
bin
```

```
obj
```

```
# User files
```

```
.vscode
```

```
.idea
```

```
# Temp files
```

```
*.log
```

```
# Secrets
```

```
.env
```




.dockerignore

```
# Downloaded content

# Built content
bin
obj
# User files
.vscode
.idea
# Temp files
*.log
# Secrets
.env
# Non production files
```



.dockerignore

```
# Downloaded content

# Built content
bin
obj
# User files
.vscode
.idea
# Temp files
*.log
# Secrets
.env
# Non production files
appsettings.Development.json
**/launchSettings.json
```



Dockerfile

```
1 FROM  
2 COPY  
3 RUN  
4 CMD
```



Dockerfile

```
1 FROM mcr.microsoft.com/dotnet/sdk:10.0-alpine
2 COPY . .
3 RUN dotnet restore
4 RUN dotnet build -c Release
5 RUN dotnet test -c Release
6 RUN dotnet publish -c Release -o /dist
7 CMD
```



Dockerfile

```
1 FROM mcr.microsoft.com/dotnet/sdk:10.0-alpine
2 COPY . .
3 RUN dotnet restore
4 RUN dotnet build -c Release
5 RUN dotnet test -c Release
6 RUN dotnet publish -c Release -o /dist
7 ENV ASPNETCORE_URLS=http://+:8080
8 ENV ASPNETCORE_ENVIRONMENT=Production
9 EXPOSE 8080
10 CMD ["dotnet", "byoc.dll"]
```



Dockerfile

```
1 FROM mcr.microsoft.com/dotnet/sdk:10.0-alpine AS build
2 WORKDIR /src
3 COPY . .
4 RUN dotnet restore
5 RUN dotnet build -c Release
6 RUN dotnet test -c Release
7 RUN dotnet publish -c Release -o /dist
8
9 FROM mcr.microsoft.com/dotnet/aspnet:10.0-alpine
10 ENV ASPNETCORE_URLS=http://+:8080
11 ENV ASPNETCORE_ENVIRONMENT=Production
12 EXPOSE 8080
13 WORKDIR /app
14 COPY --from=build /dist .
15 CMD ["dotnet", "byoc.dll"]
```



Dockerfile

```
1 FROM mcr.microsoft.com/dotnet/sdk:10.0-alpine AS build
2 WORKDIR /src
3 COPY byoc.csproj .
4 RUN dotnet restore
5 COPY . .
6 RUN dotnet build -c Release
7 RUN dotnet test -c Release
8 RUN dotnet publish -c Release -o /dist
9
10 FROM mcr.microsoft.com/dotnet/aspnet:10.0-alpine
11 ENV ASPNETCORE_URLS=http://+:8080
12 ENV ASPNETCORE_ENVIRONMENT=Production
13 EXPOSE 8080
14 WORKDIR /app
15 COPY --from=build /dist .
16 CMD ["dotnet", "byoc.dll"]
```



Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:10.0-alpine AS build
WORKDIR /src
COPY byoc.csproj .
RUN dotnet restore
COPY . .
RUN dotnet build -c Release
RUN dotnet test -c Release
RUN dotnet publish -c Release -o /dist

FROM mcr.microsoft.com/dotnet/aspnet:10.0-alpine
ENV ASPNETCORE_URLS=http://+:8080
ENV ASPNETCORE_ENVIRONMENT=Production
EXPOSE 8080
WORKDIR /app
COPY --from=build /dist .
CMD ["dotnet", "byoc.dll"]
```


Operations





CNCF

Meet us in Atlanta for KubeCon + CloudNativeCon North America · Nov 10-13 · [REGISTER TODAY](#)

[About](#)[Projects](#)[Training](#)[Community](#)[Blog & News](#)[Join](#)

WHO WE ARE

The Cloud Native Computing Foundation (CNCF) hosts critical components of the global technology infrastructure.

We bring together the world's top developers, end users, and vendors and run the largest open source developer conferences. CNCF is part of the nonprofit [Linux Foundation](#).



OUR CHARTER

"CNCF's mission is to make cloud native computing ubiquitous..."

USEFUL LINKS

- Read about recent work and accomplishments in the [CNCF Annual Report](#)
- View our [FAQ](#)



<https://cncf.io>



Kubernetes (k8s)



kubernetes

Kubernetes is an open-source system for automating deployment, scaling, and management of containerized applications

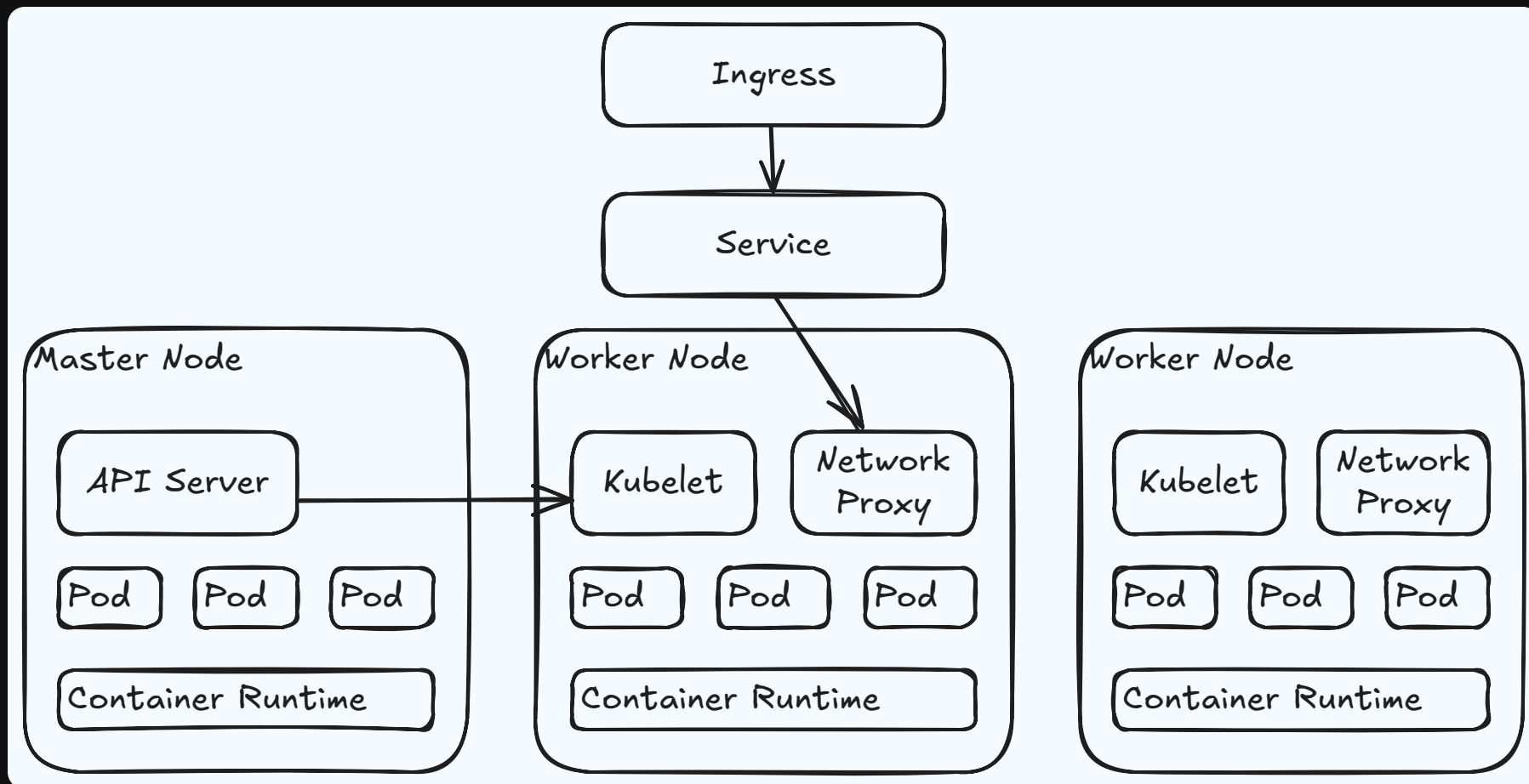
Kubernetes was accepted to CNCF on March 10, 2016 at the **Incubating** maturity level and then moved to the **Graduated** maturity level on March 6, 2018.

[VISIT PROJECT WEBSITE](#)



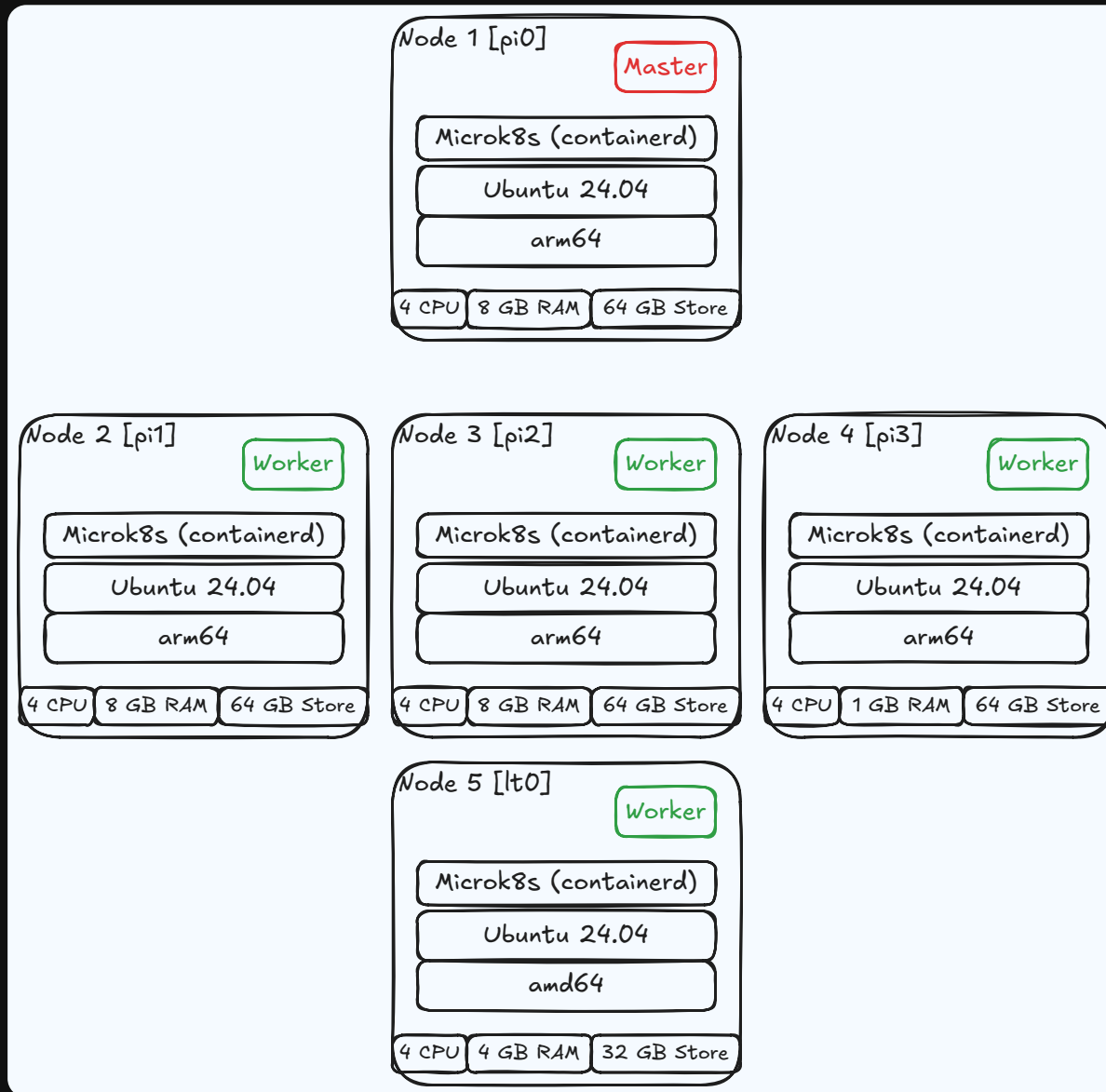


Kubernetes Architecture



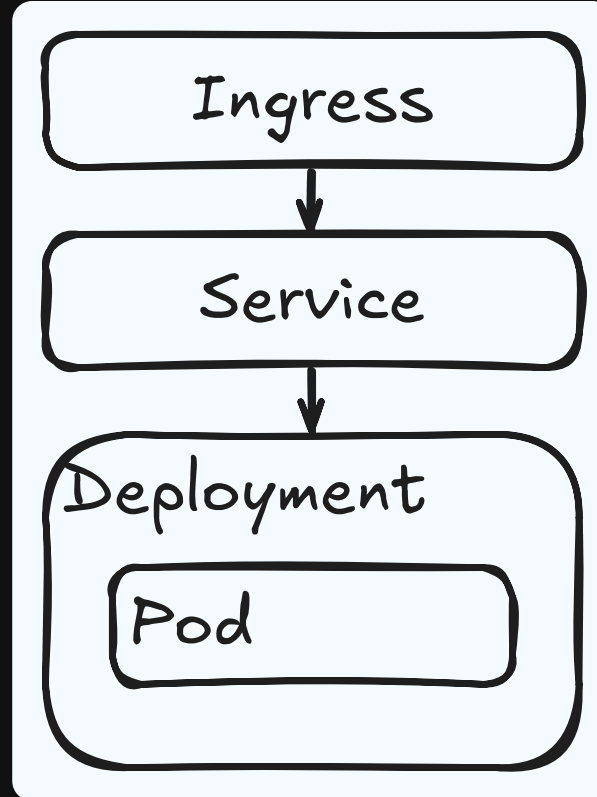


Kubernetes Cluster





Kubernetes Deployment





Kubernetes manifest

k8s.yml - Deployment

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   namespace: crayon
5   name: build-your-own-cloud
6 spec:
7   replicas: 1
8   selector:
9     matchLabels:
10      app: build-your-own-cloud
11   template:
12     metadata:
13       labels:
14         app: build-your-own-cloud
15         version: IMAGE_LABEL
16     spec:
17       imagePullSecrets:
18         - name: regcred
19       containers:
20         - name: build-your-own-cloud
21           image: ghcr.io/punsvikcloud/build-your-own-cloud:IMAGE_LABEL
22           imagePullPolicy: Always
23           resources:
24             limits:
25               memory: "128Mi"
26               cpu: "500m"
27           ports:
28             - containerPort: 8080
```



Kubernetes manifest

k8s.yml - Service

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   namespace: crayon
5   name: crayon-service
6 spec:
7   selector:
8     app: build-your-own-cloud
9   ports:
10    - port: 8080
```




Kubernetes manifest

k8s.yml - Ingress

```
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    namespace: crayon
5    name: crayon-ingress
6    labels:
7      name: crayon-ingress
8    annotations:
9      cert-manager.io/cluster-issuer: lets-encrypt
10 spec:
11   ingressClassName: nginx
12   tls:
13     - hosts:
14       - crayon.punsvik.net
15       secretName: tls-secret
16   rules:
17     - host: crayon.punsvik.net
18       http:
19         paths:
20           - pathType: Prefix
21             path: "/"
22             backend:
23               service:
24                 name: crayon-service
25                 port:
26                   number: 8080
```



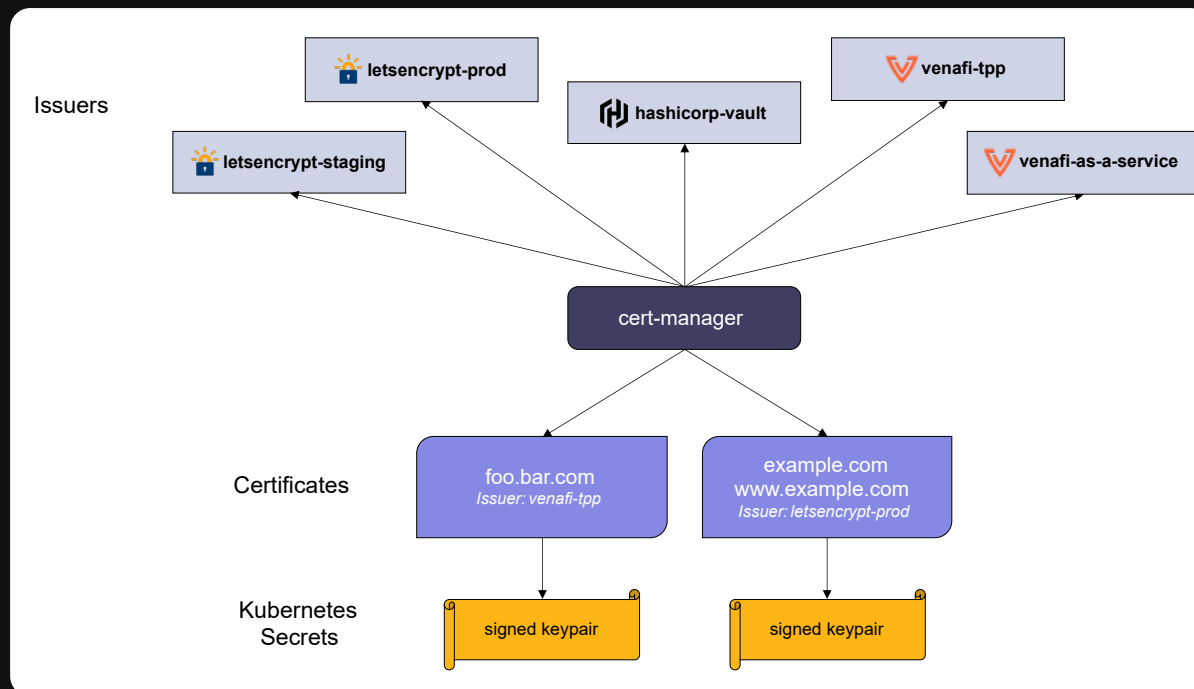
Kubernetes manifest

k8s.yml - Ingress

```
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    namespace: crayon
5    name: crayon-ingress
6    labels:
7      name: crayon-ingress
8    annotations:
9      cert-manager.io/cluster-issuer: lets-encrypt
10 spec:
11   ingressClassName: nginx
12   tls:
13     - hosts:
14       - crayon.punsvik.net
15       secretName: tls-secret
16   rules:
17     - host: crayon.punsvik.net
18       http:
19         paths:
20           - pathType: Prefix
21             path: "/"
22             backend:
23               service:
24                 name: crayon-service
25                 port:
26                   number: 8080
```



cert-manager



<https://github.com/cert-manager/cert-manager>



Cloudflare DNS

Cloudflare interface showing DNS management for **nctest.info**. The interface includes a sidebar with navigation options and a main content area for DNS records.

Navigation Sidebar:

- Overview
- Analytics
- DNS** (highlighted with a green box and arrow 1)
- Email (Beta)
- SSL/TLS
- Firewall
- Access
- Speed
- Caching
- Workers
- Rules
- Network
- Traffic
- Custom Pages
- Apps

DNS Management for nctest.info:

A few more steps are required to complete your setup. [Show](#)

Search DNS Records Advanced + Add record (highlighted with a green box and arrow 2)

test.nctest.info points to 198.187.31.46 and has its traffic proxied through Cloudflare.

Form Fields (highlighted with a green box and arrow 3):

- Type: A
- Name (required): test
- IPv4 address (required): 198.187.31.46
- Proxy status: ☒ Proxied
- TTL: Auto

Use @ for root

Save Button (highlighted with a green box and arrow 4): Save

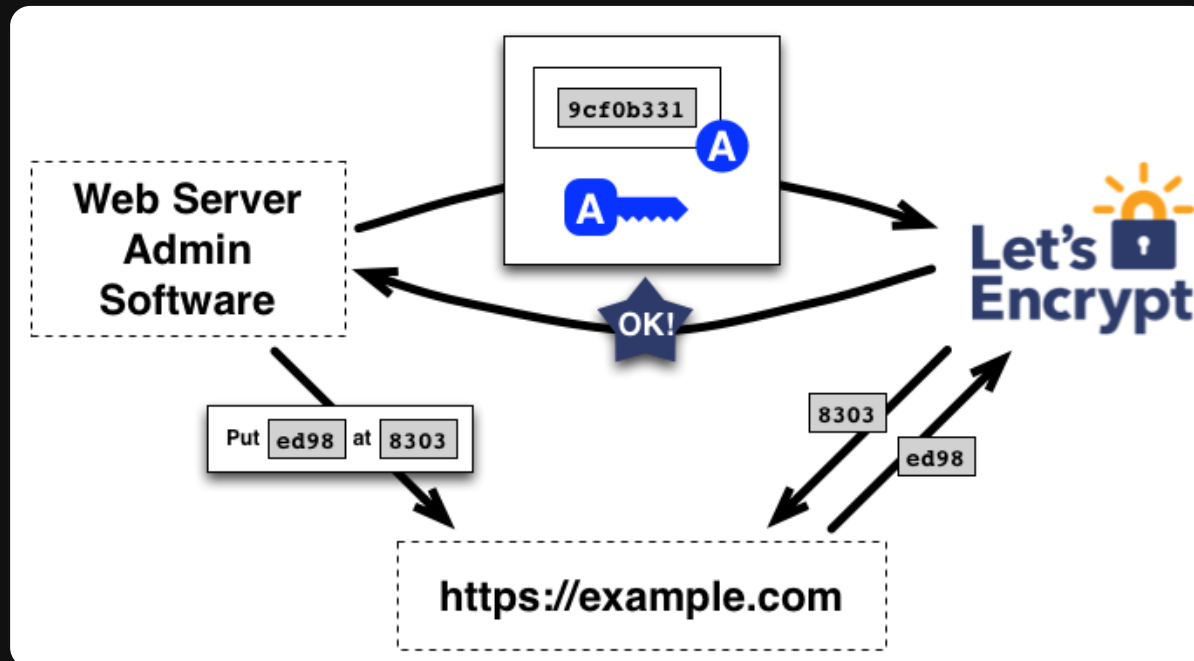
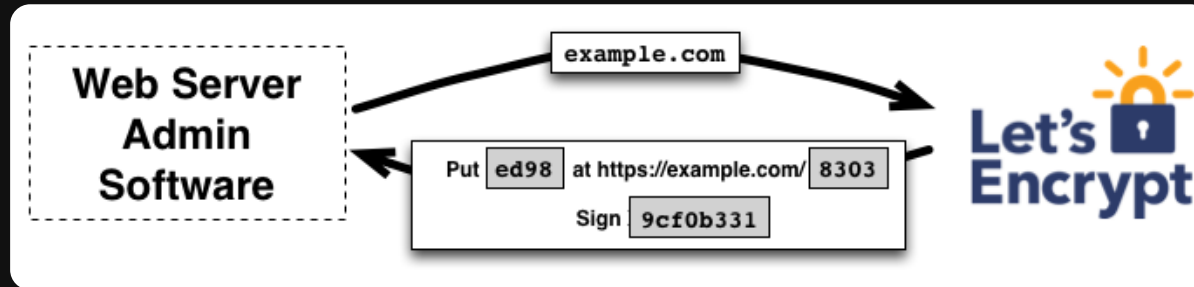
Table of DNS Records (highlighted with a green box and arrow 5):

Type	Name	Content	Proxy status	TTL	Actions
A	autoconfig	198.187.31.46	Proxied	Auto	Edit
A	autodiscover	198.187.31.46	Proxied	Auto	Edit
A	cpanel	198.187.31.46	DNS only	Auto	Edit
A	cpcalendars	198.187.31.46	Proxied	Auto	Edit
A	cpcontacts	198.187.31.46	Proxied	Auto	Edit



Let's Encrypt

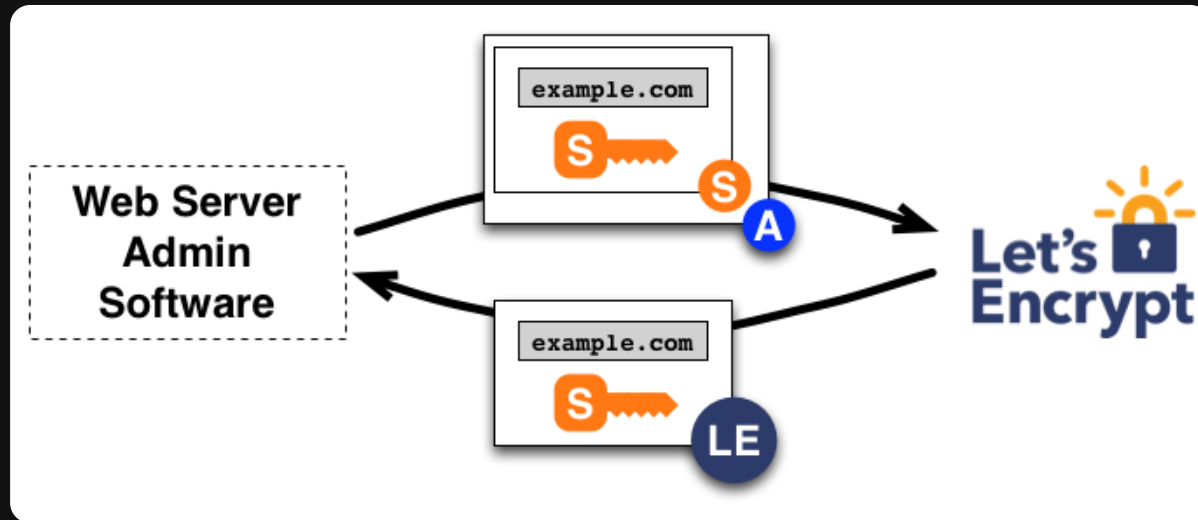
Domain Validation



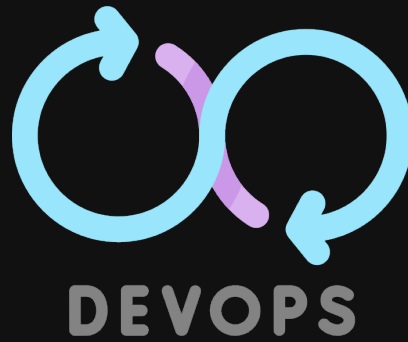


Let's Encrypt

Certificate Issuance



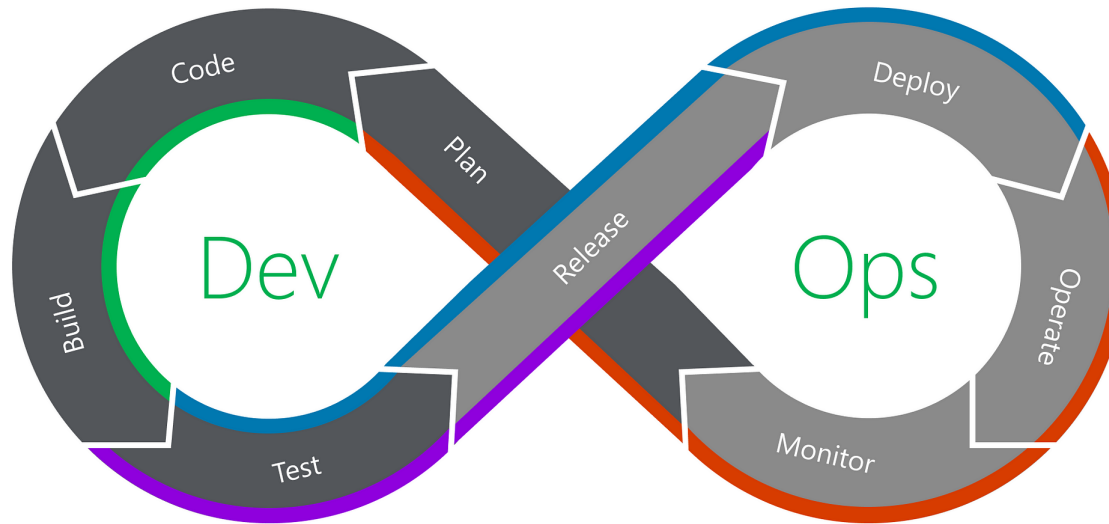
DevOps





DevOps

Communication, Collaboration and Security

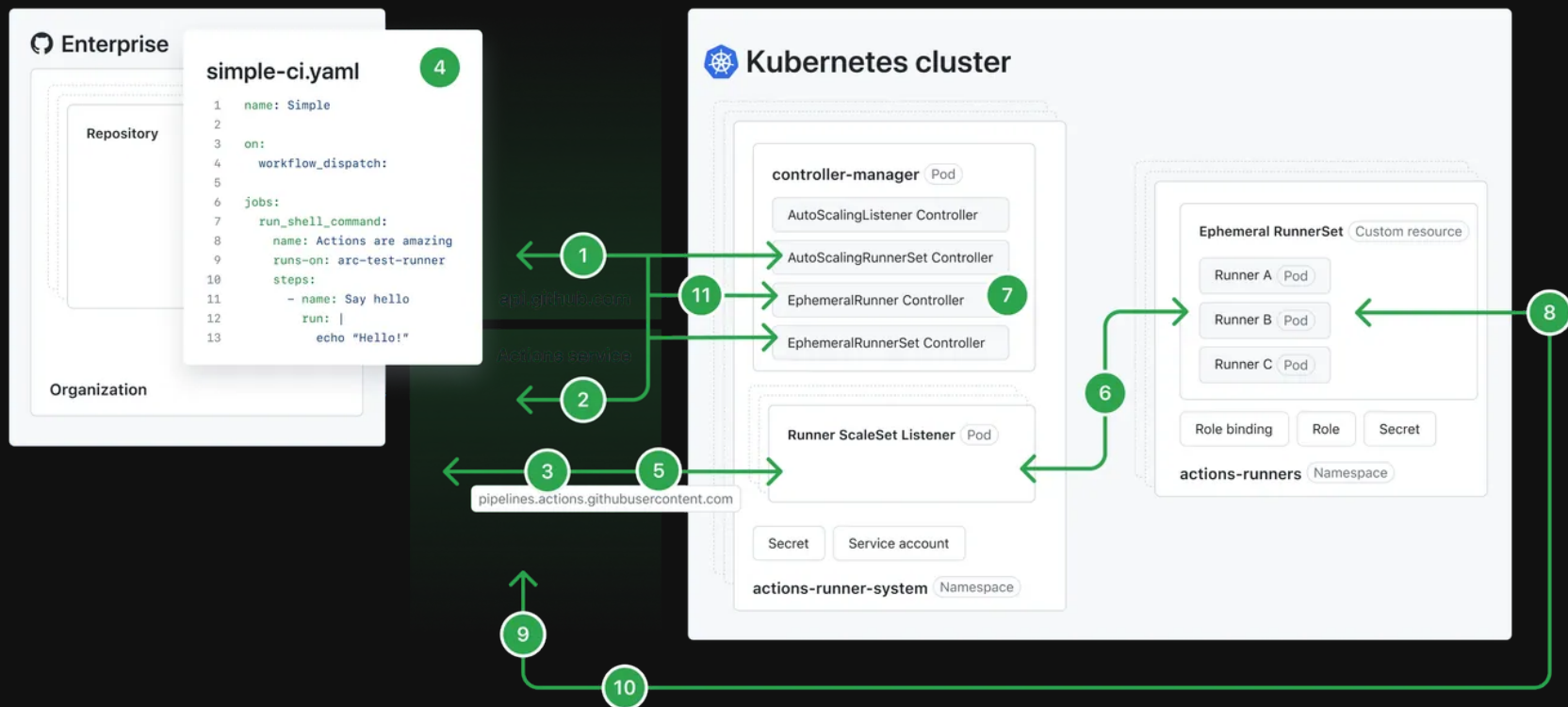


- Continuous Integration (CI)
- Continuous Deployment (CD)
- Continuous Delivery (CD)
- Continuous Feedback (CF)





GitHub Action Runners





GitHub Action - Runners

**Punsvik Cloud**
Organization [Switch settings context](#) ▼

Go to your organization profile

 General

 Policies ▼

Access

 Billing and licensing ▼

 Organization roles ▼

 Repository roles

 Member privileges

 Import/Export

 Moderation ▼

Code, planning, and automation

 Repository ▼

Runners

Includes all runners across self-hosted and GitHub-hosted runners.

Host your own runners and customize the environment used to run jobs in your GitHub Actions workflows. Runners added to this organization can be used to process jobs in multiple repositories in your organization. [Learn more about self-hosted runners.](#)

New runner ▼

Runners	Status
 Standard GitHub-hosted runners Ready-to-use runners managed by GitHub. Learn more.	● 0 active jobs
 self-hosted self-hosted Runner group: <u>Default</u>	● Online

GitHub Action - Secrets


Punsvik Cloud
 Organization [Switch settings context](#)

Go to your organization profile

 General
  Policies

Access

 Billing and licensing
  Organization roles
  Repository roles
  Member privileges
  Import/Export
  Moderation

Code, planning, and automation

 Repository
  Codespaces
  Planning
  Copilot
  Actions
  Models

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for non-sensitive data. [Learn more about variables](#).

Anyone with collaborator access to the repositories with access to a secret or variable can use it for Actions. They are not passed to workflows that are triggered by a pull request from a fork.

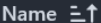
Organization secrets and variables cannot be used by private repositories with your plan. Please consider [upgrading your plan](#) if you require this functionality.

Secrets

Variables

Organization secrets

New organization secret

Name 	Visibility	Last updated	
 GHCR_PULL_TOKEN	Public repositories	last year	 
 KUBERNETES_SECRET	Public repositories	last year	 
 KUBERNETES_URL	Public repositories	last year	 



GitHub Action

.github/workflows/deploy.yml - Metadata

```
1 name: Docker Build & Kubernetes Run
2
3 on:
4   push:
5     branches:
6       - main
7   workflow_dispatch:
8
9 env:
10  REGISTRY: ghcr.io
11  PLATFORMS: linux/arm64
12  K8S_NAMESPACE: crayon
13
14 jobs:
15   build:
16     runs-on: self-hosted
17     permissions:
18       contents: read
19       packages: write
20       attestations: write
21       id-token: write
22     steps:
23       ...
```



GitHub Action

.github/workflows/deploy.yml - Build

```
1 - name: Checkout Code
2   uses: actions/checkout@v4
3 - name: Set up Docker Buildx
4   uses: docker/setup-buildx-action@v3
5   with:
6     driver: kubernetes
7 - name: Docker Login
8   uses: docker/login-action@v3.3.0
9   with:
10    registry: ${ env.REGISTRY }
11    username: ${ github.actor }
12    password: ${ secrets.GITHUB_TOKEN }
13    logout: true
```

```
1 - name: Extract metadata (tags, labels) for Docker
2   id: meta
3   uses: docker/metadata-action@v5
4   with:
5     images: ${ env.REGISTRY }/${ github.repository }
6     flavor: latest=true
7     tags: |
8       type=ref,event=branch
9       type=raw,value=latest,enable=${ is_default_branch }
10      type=raw,value=${ github.sha }
11 - name: Build and push Docker image
12   id: push
13   uses: docker/build-push-action@v6
14   with:
15     context: .
16     push: true
17     platforms: linux/arm64
18     tags: ${ steps.meta.outputs.tags }
19     labels: ${ steps.meta.outputs.labels }
```



GitHub Action

.github/workflows/deploy.yml - Deploy

```
1  deploy:
2    runs-on: self-hosted
3    needs: build
4    permissions:
5      id-token: write
6      actions: read
7      contents: read
8    steps:
9      - name: Checkout Code
10        uses: actions/checkout@v4
11
12      - name: Log in to the Container registry
13        uses: docker/login-action@v3
14        with:
15          registry: ${ env.REGISTRY }
16          username: ${ github.actor }
17          password: ${ secrets.GITHUB_TOKEN }
18
19      - name: Set K8s context
20        uses: azure/k8s-set-context@v4
21        with:
22          method: service-account
23          k8s-url: ${ secrets.KUBERNETES_URL }
24          k8s-secret: ${ secrets.KUBERNETES_SECRET }
```

```
1    - name: Set kubectl
2      uses: azure/setup-kubectl@v4
3      id: install
4      with:
5        version: "v1.29.9"
6    - name: Set imagePullSecret
7      uses: azure/k8s-create-secret@v4
8      id: create-secret
9      with:
10        namespace: ${ env.K8S_NAMESPACE }
11        secret-name: regcred
12        container-registry-url: ${ env.REGISTRY }
13        container-registry-username: ${ github.actor }
14        container-registry-password: ${ secrets.GHCR_PULL_TOKEN }
15    - name: Set Label
16      run: sed -i'' -e 's/IMAGE_LABEL/${ github.sha }/g' k8s.yml
17    - name: Deploy application
18      uses: azure/k8s-deploy@v5
19      with:
20        action: deploy
21        manifests: k8s.yml
22        namespace: ${ env.K8S_NAMESPACE }
23        images: ${ env.REGISTRY }/${ github.repository }:${ github.sha }
```



Action waiting

```
Requested labels: self-hosted  
Job defined at: PunsvikCloud/build-your-own-cloud/.github/workflows/docker-kubernetes.yml@refs/heads/main  
Waiting for a runner to pick up this job...  
Job is about to start running on the runner: self-hosted
```


Action details

Triggered via push 5 minutes ago

Status

Total duration

Artifacts

 bjornkpu pushed  [ca92562](#) main

Success

5m 41s

—

docker-kubernetes.yml
on: push

 build

3m 2s



 deploy

1m 31s

Packages

PunsvikCloud



[Overview](#) [Repositories 7](#) [Projects](#) [Packages](#) [Teams](#) 

Type: All 

 Search packages...

Visibility: All 

Sort by: Most downloads 

 1 package

 **build-your-own-cloud** Private

Published on Nov 12, 2025 by [Punsvik Cloud](#) in [PunsvikCloud/build-your-own-cloud](#)

 2

[Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#)

Do not share my personal information

 © 2025 GitHub, Inc.

Package Details

 **build-your-own-cloud** `ca92562d25b9ff27f6ece9a9dd79ec19f7bbd4a5` Private Latest

Installation OS / Arch 2

[Learn more about packages](#)

 Install from the command line

```
$ docker pull ghcr.io/punsvikcloud/build-your-own-cloud:ca92562d25b9ff27f6ece9a9dd79ec19f7bbd4a5
```

Recent tagged image versions

latest `ca92562d25b9ff27f6ece9a9dd79ec19f7bbd4a5` main

 1

Published about 5 minutes ago · Digest ...

[View and manage all versions](#)

Details

 PunsvikCloud

 build-your-own-cloud

☆ 0 stars

Last published

5 min ago

Issues

0

Total downloads

2

Contributors 1



bjornkpu Bjørn Kristian Pu...

